

EEL 4930/5934 - System-on-Chip Design

Final Project

Hashini Fernando, Marc Martorell, Leandro Sanchez
University of Florida

I. TASK 2.1 - PONG GAME IMPLEMENTATION

This task focuses on the implementation of a hardware-accelerated Pong game using the NEORV32 soft-core processor. The design integrates video output, user input, and real-time game logic within a custom system-on-chip architecture.

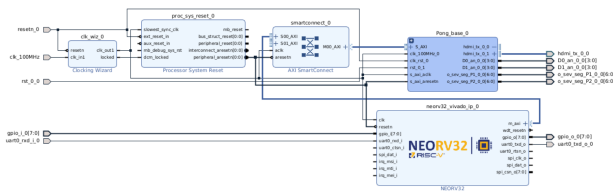


Fig. 1. NEORV32-based system architecture for Pong game implementation.

The system integrates a Pong IP comprising a VGA controller for video output and dedicated hardware logic for ball position computation and score tracking for two-players. Player scores are displayed via a seven-segment display interface. At the hardware level, paddle inputs are captured through GPIO buttons and processed by the NEORV32 soft core processor. The communication between the processor and Pong IP is achieved through an AXI-based Interconnect utilizing AXI smart-connect and AXI GPIO peripherals.

At the software level, the Pong IP is mapped to a fixed base address (0x40000000), enabling direct access from the processor. User input is captured through the GPIO interface, where four push buttons and one onboard switch are mapped to specific bit positions corresponding to paddle control signals for two players and the game START/PAUSE command. These inputs are encoded into a control word and written to Pong IP control register at offset 0x0, allowing the hardware to update paddle positions and game state. Additionally, an initialization step was added to configure the AXI GPIO direction register at offset 0x4 to ensure proper IO behavior as output for the interface.

The firmware periodically samples the NEORV32 GPIO input port and updates the Pong IP at a fixed interval using a software delay. A simple edge-detection mechanism is implemented for the START signal to detect transitions and provide debug feedback via the UART interface [1].

Key challenges in this task was ensuring correct integration between the processor and AXI peripherals. In particular, verifying correct register offsets and proper configuration of the AXI GPIO direction registers was necessary to ensure correct IO behavior. These challenges were addressed through

iterative debugging, register level verification and validation of hardware-software interactions [2].

Overall, this task demonstrates real-time embedded system design, highlighting the integration of processor control with hardware peripherals to achieve interactive functionality.

II. TASK 2.2 - CRYPTOGRAPHIC ACCELERATION IMPLEMENTATION

This project implements and evaluates cryptographic acceleration techniques on the NEORV32 soft-core processor, focusing on AES-128 and GIFT-128. Three execution models were explored for AES: software (SW), custom function subsystem hardware acceleration (HW), and RISC-V cryptographic instruction set extensions (ISA). In contrast, GIFT-128 was implemented and validated in software only due to hardware complexity and timing constraints.

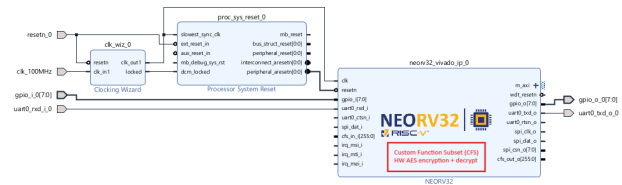


Fig. 2. NEORV32-based system architecture with AES hardware accelerator and peripherals.

The AES hardware accelerator was integrated via the Custom Functions Subsystem (CFS), enabling direct interaction between the processor and a dedicated encryption datapath. Functional correctness was verified using standard AES test vectors, ensuring consistency across SW, HW, and ISA implementations.

Performance was evaluated using cycle counts averaged over multiple runs. The software implementation required approximately 13.8 million cycles, while the hardware-accelerated version completed in approximately 130 thousand cycles, corresponding to a speedup of $\sim 105\times$. The ISA-based implementation required approximately 848 thousand cycles, achieving a $\sim 16\times$ speedup over software. These results highlight the substantial benefit of dedicated hardware acceleration, with ISA extensions providing a moderate improvement with significantly lower implementation complexity.

GIFT-128 was implemented in software and validated against known test vectors. While a hardware version was

```

CRYPTO
TRNG OK

AES
K 000102030405060708090a0b0c0d0e0f
P 00112233445566778899aabbccddeeff
S 69c4e0d86a7b0430d8cdb78070b4c55a
H 69c4e0d86a7b0430d8cdb78070b4c55a
I 69c4e0d86a7b0430d8cdb78070b4c55a
E 69c4e0d86a7b0430d8cdb78070b4c55a
A-SW PASS
A-HW PASS
A-IS PASS
A-SH PASS
A-SI PASS
A-DC PASS
A cyc sw/hw/is 13812624 130524 847924
A spd hw 105
A spd is 16
A-RND PASS

GFT
S0 cd0bd738388ad3f668b15a36ceb6ff92
E0 cd0bd738388ad3f668b15a36ceb6ff92
G0 PASS
G0 DEC PASS
S1 181691d526a3c678dd28fb9f1ce31fdd
E1 181691d526a3c678dd28fb9f1ce31fdd
G1 PASS
G1 DEC PASS
G cyc sw 95125124
G-RND PASS
DONE

```

Fig. 3. UART output showing AES validation and performance benchmarking results.

initially explored, challenges related to bit-sliced state representation, permutation mapping, and round key scheduling resulted in deviations from the specification. Due to these complexities, the hardware implementation was excluded from the final design, and focus was placed on software correctness and performance evaluation.

A True Random Number Generator (TRNG) module was incorporated to generate random plaintext and key inputs, enabling dynamic validation of encryption consistency. This approach provided additional verification beyond fixed test vectors.

Key challenges included instruction memory constraints (ROM overflow), UART formatting limitations, and debugging low-level bit manipulations in the GIFT hardware design. These issues were addressed through code optimization, modular testing, and iterative verification.

Overall, the project demonstrates that hardware acceleration provides the highest performance gains for cryptographic workloads, while ISA extensions offer a balanced trade-off between performance improvement and implementation com-

plexity in embedded systems.

III. TASK 2.3 - FREE RTOS INTEGRATION

This task focuses on implementing a real-time operating system for the NEORV32 while also using the DDR3 RAM on the board. Additional components were implemented to ensure proper operating system functionality, including a watchdog timer, dual-core threading, and core-local interrupts. The operating system of choice was FreeRTOS due to its ease of use, extensive documentation, and pre-existing examples.

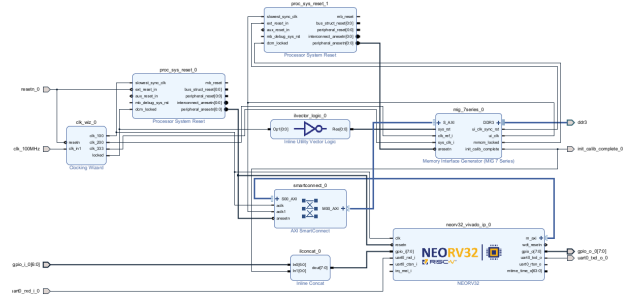


Fig. 4. NEORV32-based system architecture with FreeRTOS support and DDR3 Memory

A. DDR3 Memory

The DDR3 was implemented using the Memory Interface Generator 7 Series provided by Vivado. The MIG 7 Series translates data between the AXI interface used by the NEORV32 and the physical DDR3 signals. Although DDR3 can be used as an external RAM, the processor still requires a small amount of internal data memory to boot up successfully. Afterwards, the DDR3 can be used starting at address (0x88000000).

The most challenging aspect of the DDR3 implementation was synchronizing the reset across the board, processor, and memory controller. The design requires three distinct clock frequencies: 100 MHz for general logic, 200 MHz for the MIG reference clock, and 333.333 MHz for the DDR3 interface. The MIG's asynchronous reset is driven by the 166.667 MHz clock (half the DDR3 rate) and is also fed into one of the SmartConnect's clock domains. The SmartConnect then uses the NEORV32 clock as its second domain, allowing it to safely propagate the reset signal across the clock boundary. Finally, the DDR3 system reset is gated by the Clocking Wizard's locked signal, ensuring the memory interface only comes out of reset once all clocks have stabilized.

B. FreeRTOS

The FreeRTOS is implemented using a simple producer-consumer model with an IRQ to capture button inputs and add a button press event to the main processing queue.

```
typedef struct {
    uint32_t pin;
    uint32_t edge;
    // 1 = rising (press),
    // 0 = falling (release)
} ButtonEvent_t;
```

The main challenge with this system was debouncing the button using the IRQ. Without debouncing, the IRQ can be triggered multiple times within a single press. A delay in the processing thread helps, but the IRQ can only see one edge at a time, so we also need to alternate the IRQ between rising and falling edges. This setup correctly detects when a button is pressed and released without needing to poll.

Additional components are needed to maintain the correct functionality of the operating system. A watchdog timer is important when the code gets stuck and needs to be reset. Every five seconds, the watchdog timer will check to see if it has been fed. There is a dedicated function that runs at a set interval called *vApplicationIdleHook*. This function is meant to run when the CPU is idle, but can also be repurposed to feed the timer. If five seconds have elapsed and not fed, the program will be restarted.

```
void vApplicationIdleHook(void) {

    // Reset Watch Dog Timer
    neorv32_wdt_feed(WDT_PASSWORD);

    // cpu wakes up on any
    // interrupt request
    neorv32_cpu_sleep();
}
```

Core local interrupts and dual-core interrupts are also necessary, and can be set up from the NEORV32 settings GUI, but have no software implementation.

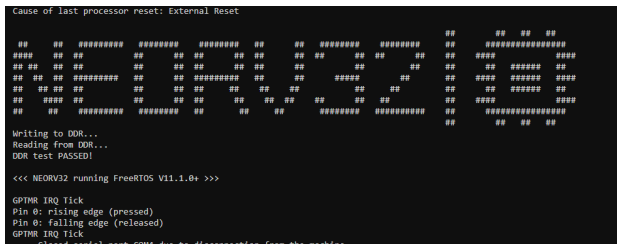


Fig. 5. NEORV32 Starting screen

REFERENCES

- [1] API Reference: File List — stnolting.github.io. <https://stnolting.github.io/neorv32/sw/files.html>. [Online].
- [2] [Data Sheet] The NEORV32 RISC-V Processor — stnolting.github.io. <https://stnolting.github.io/neorv32/>. [Online].